

CSE4067

Introduction to Blockchain Programming

Goshgar Can Ismayilov

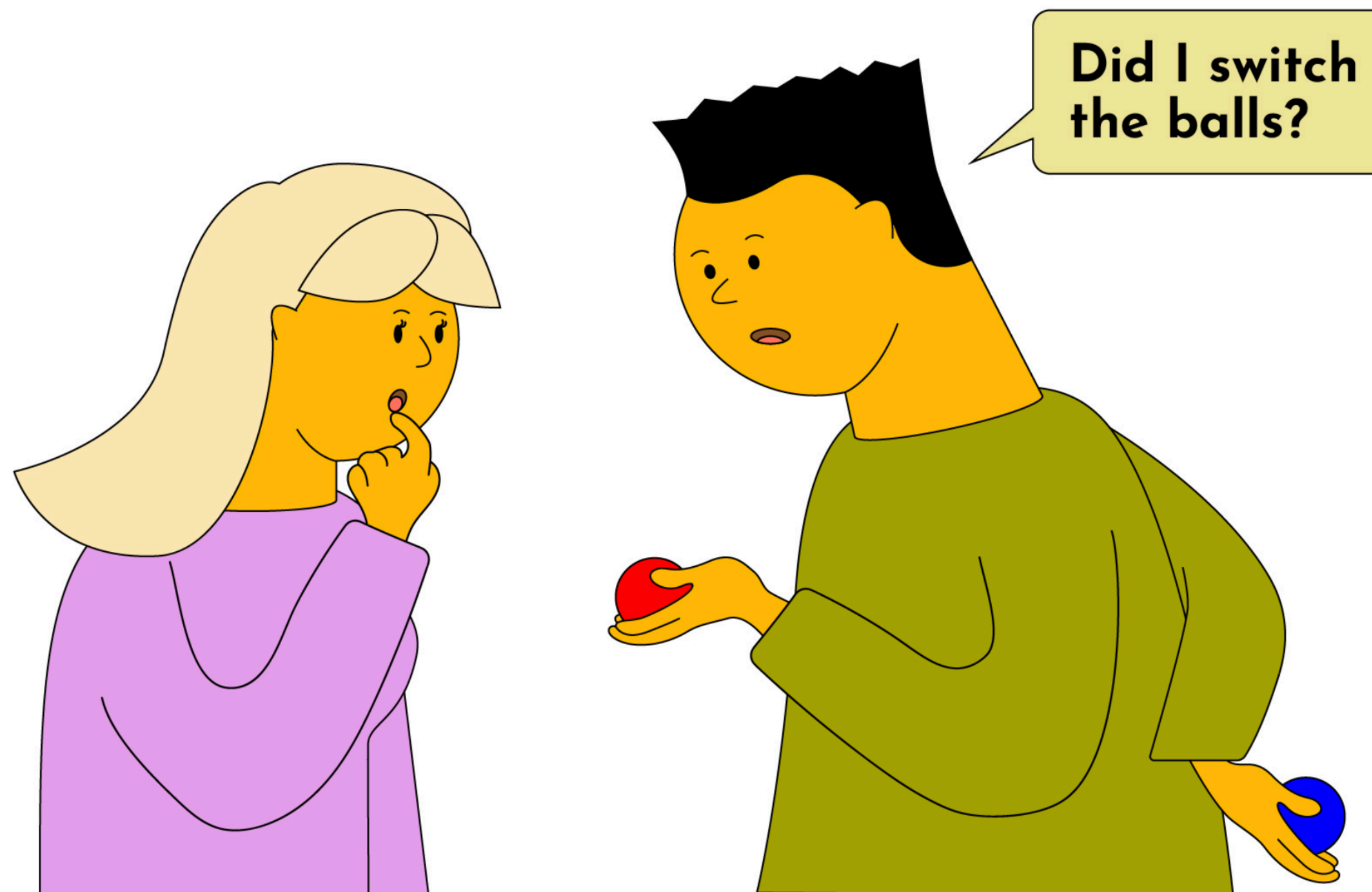
goshgar.ismayilov@boun.edu.tr

<https://www.linkedin.com/in/gismayilov/>

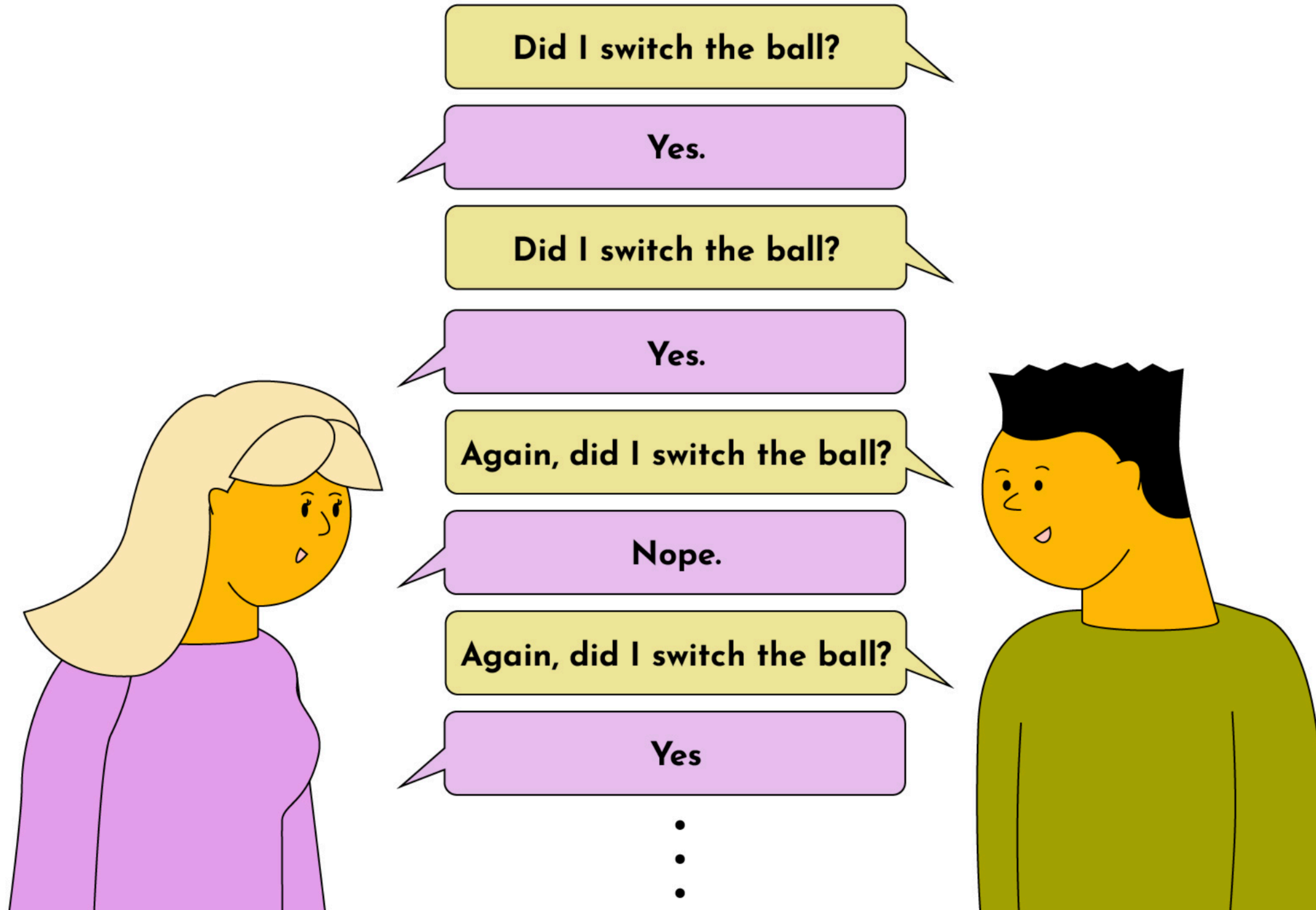
17.12.2024

Zero-Knowledge Proof: Color Blindness

Imagine Peggy has two colored balls — one is red, and the other is blue. Since Victor is colorblind, he might not believe that the balls are different colors. Peggy, who isn't colorblind, wants to prove to Victor that the balls are different colors, even though she can't actually reveal the colors.



Zero-Knowledge Proof: Color Blindness



Zero-Knowledge Proof: Marbles

1

Alice claims that two cups contain the same number of marbles



She then also offers two buckets of marbles

She claims these buckets also contain an identical number of marbles

2

Bob chooses randomly into which bucket which cup is poured
(L,L) and (R,R) or (L,R) and (R,L)



3

Bob now counts the marbles in each bucket and should find the same number in both



50% confidence after 1st game
75% confidence after 2nd game
95% confidence after 5th game

Zero-Knowledge Proof: Warheads



A ZERO-KNOWLEDGE PROTOCOL FOR NUCLEAR WARHEAD VERIFICATION

Alexander Glaser* and Robert J. Goldston**

*Princeton University

**Princeton University and Princeton Plasma Physics Laboratory

Los Alamos National Laboratory, March 19, 2015

Zero-Knowledge Proof: Warheads



CORRECTNESS OF DECLARATIONS

- Warhead Counting
Verify that numerical limit of declared items is not exceeded
- Warhead Authentication
Verify authenticity of warheads prior to dismantlement



COMPLETENESS OF DECLARATIONS

- How to make sure that no covert warheads exist outside the verification regime?

Also (very) important, but not discussed here

Zero-Knowledge Proof: Warheads



TEMPLATE-MATCHING (using active neutron interrogation)

More difficult to implement than attribute approach,
but also more robust against important diversion scenarios

Needs “golden warheads” to generate templates (reference signatures)



ZERO-KNOWLEDGE PROTOCOL

Can prove that a statement is true without revealing why it is true

If successfully implemented, no requirement for “engineered” information barrier

Robust against “curious verifier”



NON-ELECTRONIC DETECTORS

Electronic hardware and software used for detectors and/or information barriers
are hard to certify and authenticate

Technologies based on non-electronic detection and storage may offer important advantages

Zero-Knowledge Proof: Warheads

1

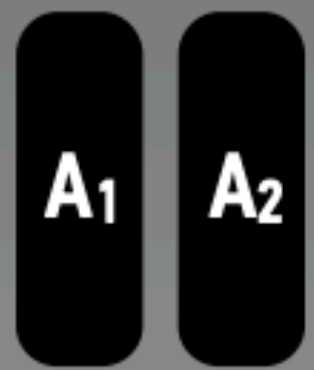
Host secretly preloads arrays of bubble detectors with “negative” radiograph of the reference item



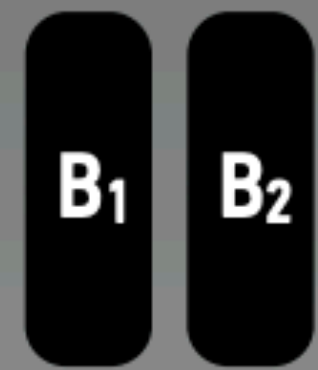
ARRAY 1



ARRAY 2



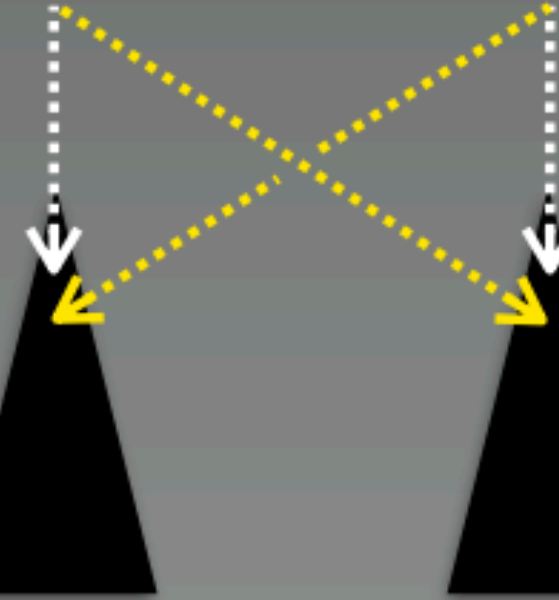
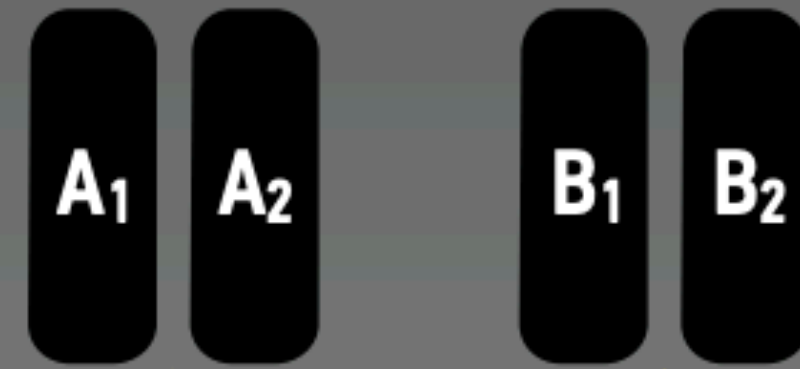
ARRAY 1



ARRAY 2

2

Inspector randomly chooses, which preloaded array to use on which item

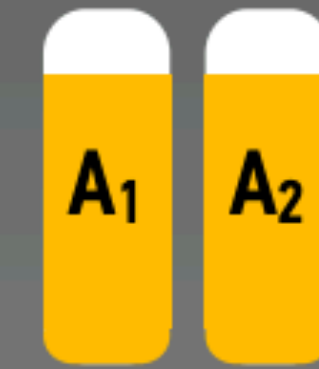


Reference item

Test item

3

After interrogation, inspector verifies that detectors in arrays contain the same bubble count



ARRAY 1



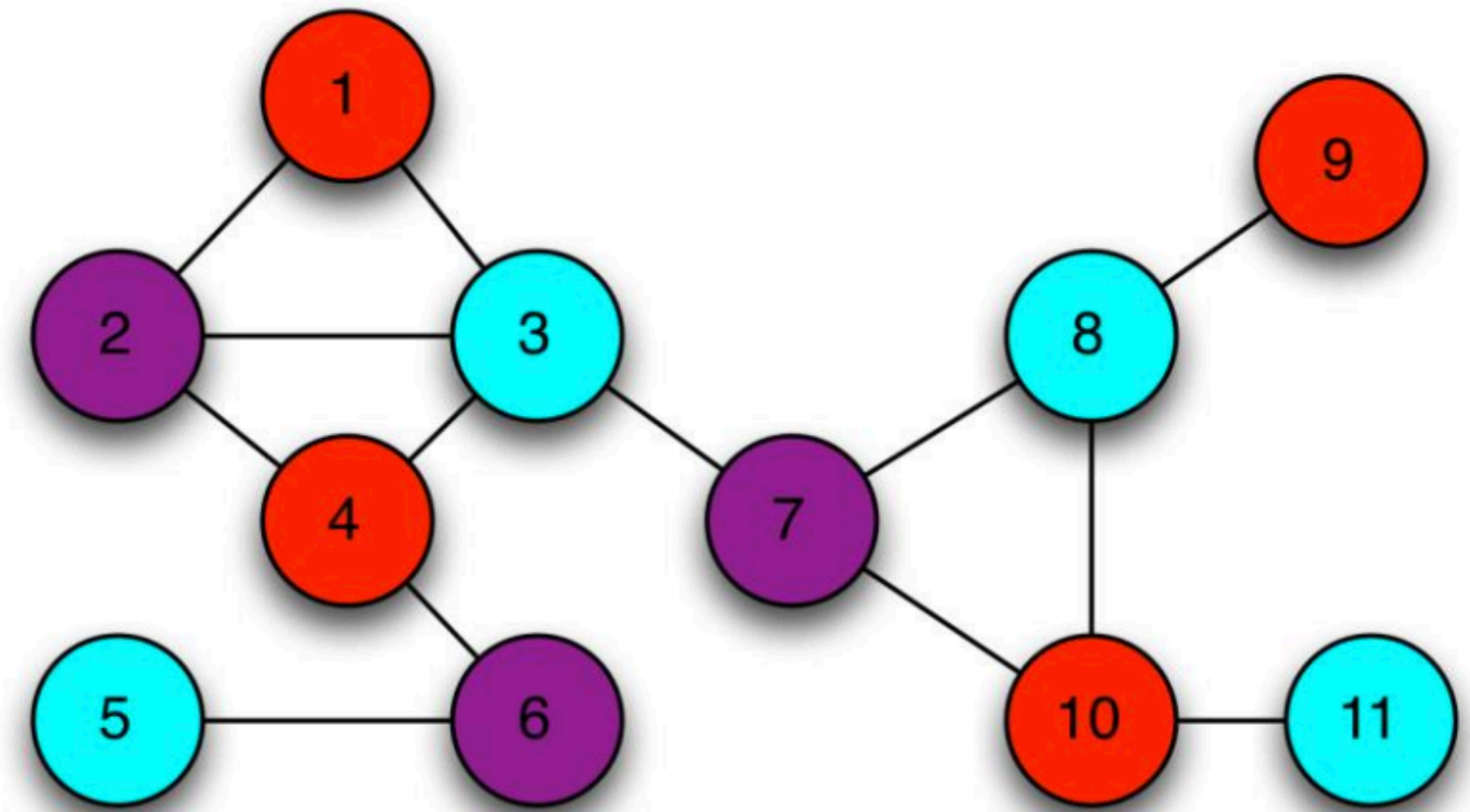
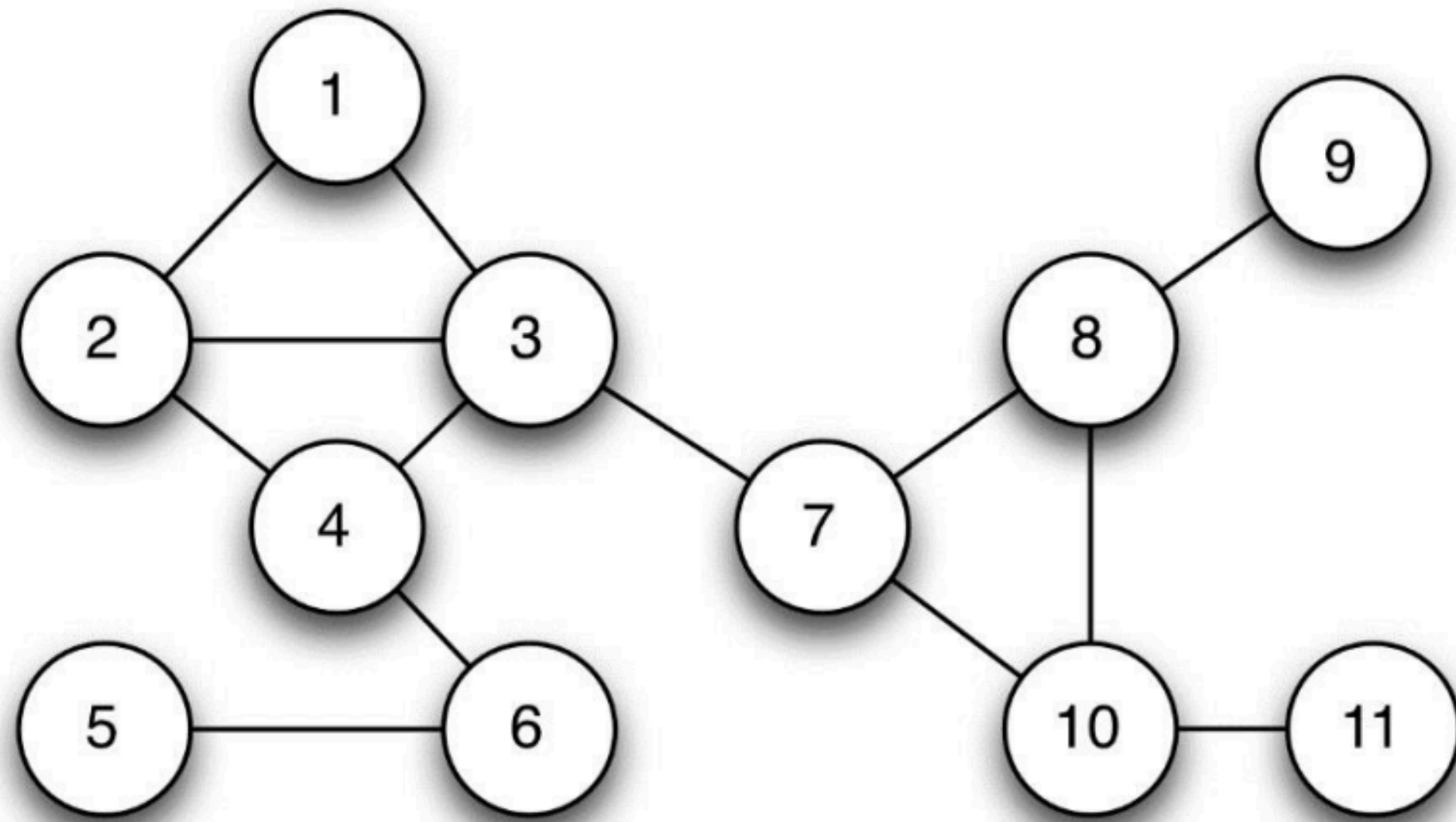
ARRAY 2

50% confidence after 1st round

...

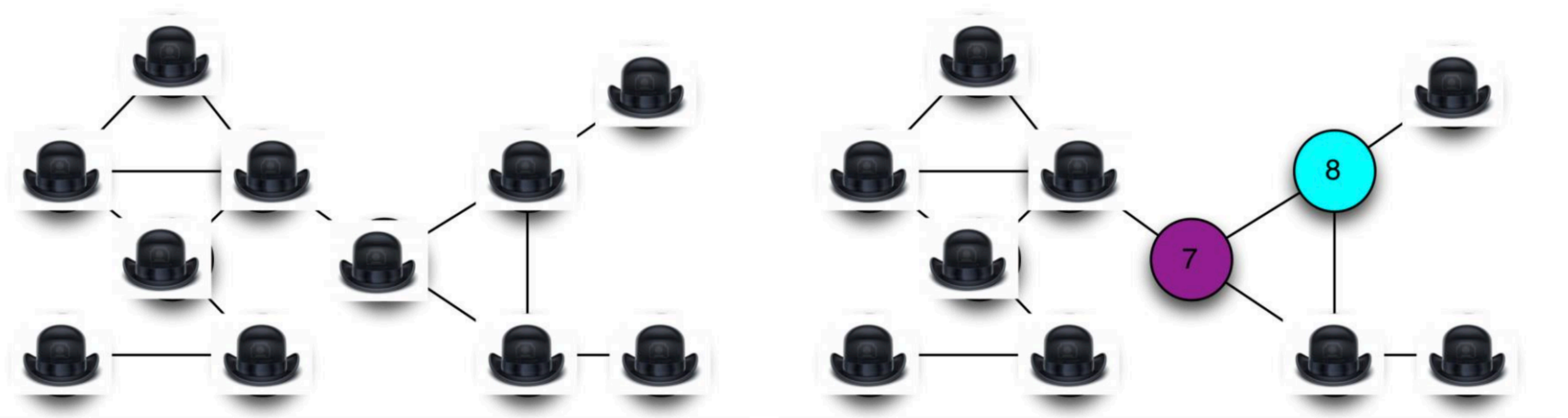
95% confidence after 5th round

Zero-Knowledge Proof: Graph 3-Coloring



complexity class: NP-Complete

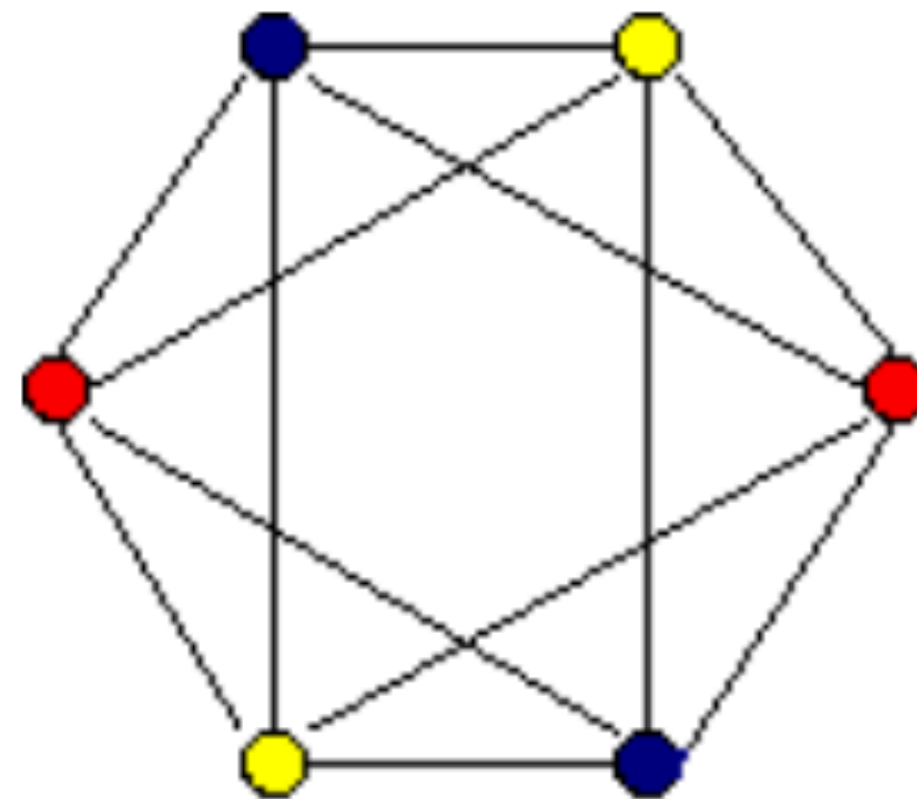
Zero-Knowledge Proof: Graph 3-Coloring



Propability of succesful cheat: $<(1-1/E)$

Zero-Knowledge Proof: Graph 3-Coloring

Definition 4.1 (Graph Three-Colorability and the language G3C [1]). A graph $G(V, E)$ is said to be 3-colorable if there exists a mapping $\phi : V \rightarrow \{1, 2, 3\}$ (called a proper coloring) such that every two adjacent vertices are assigned different colors (i.e., each $(u, v) \in E$ satisfies $\phi(u) \neq \phi(v)$). Such 3-coloring induces a partition of the vertex set of the graph to three independent sets. The language graph 3-colorability, denoted G3C, consists of the set of undirected graphs that are 3-colorable.



Zero-Knowledge Proof: Graph 3-Coloring

- (P1) The prover chooses at random an assignment of three colors to the three independent sets induced by (the proper 3-coloring) ϕ , colors the graph using this 3-coloring, and places these colors in n locked boxes each bearing the number of the corresponding vertex. More specifically, the prover chooses a permutation $\pi \in_R S_3$ (the symmetric group of 3 elements), places $\pi(\phi(i))$ in a box marked i ($\forall i \in V$), locks all boxes and sends them (without the keys) to the verifier.
- (V1) The verifier chooses at random an edge $e \in_R E$ and sends it to the prover. (Intuitively, the verifier asks to examine the colors of the endpoints of $e \in E$.)

Zero-Knowledge Proof: Graph 3-Coloring

- (P2) If $e = (u, v) \in E$, then the prover reveals the colors of u and v (to the verifier) by sending the keys to boxes u and v . Otherwise, the prover does nothing.
- (V2) The verifier opens boxes u and v using the keys received and checks whether they contain two different elements of $\{1, 2, 3\}$. If the keys do not match the boxes, or the contents violates the condition, then the verifier *rejects* and stops. Otherwise, the verifier continues to the next iteration.

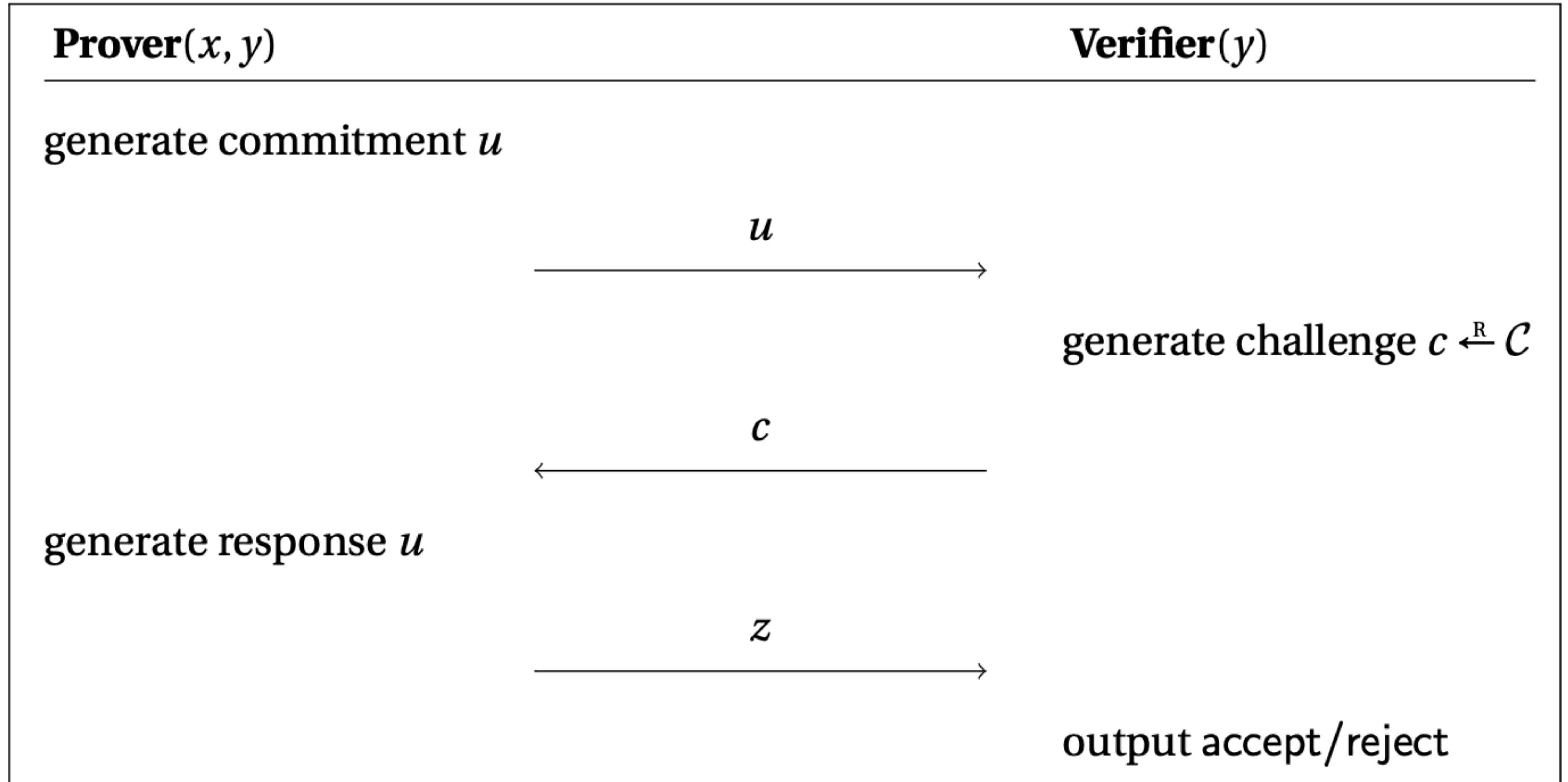
Zero-Knowledge Proof: Graph 3-Coloring

Completeness: Let $G \in \text{G3C}$. Then any pair of boxes u and v corresponding to some edge of G will certainly be colored differently. Therefore, an honest verifier will complete all m^2 iterations and *accept* with probability 1.

Soundness: Let $G \notin \text{G3C}$. Then at least 1 of the m edges of G is not properly colored. Therefore, an honest verifier will *reject* with probability at least $\frac{1}{m}$ in each round.

Zero-knowledge: The only information revealed in each round is $\pi(\phi(u))$ and $\pi(\phi(v))$ where $(u, v) = e \in_R E$, $\pi \in_R S_3$. Due to the random selection of π , a simulator which simply outputs a pair of integers (i, j) , $i, j \in_R \{1, 2, 3\}$ with $i \neq j$, is computationally indistinguishable from interaction with the prover. $\square$

Sigma Protocol



Fiat-Shamir Protocol

Integer Factorization

Protocol 5.1 (Fiat-Shamir Identification Protocol [4]).

Initialization

1. A trusted center T selects and publishes an RSA-like modulus $n = pq$ but keeps the primes p and q secret.
2. The prover selects a secret s coprime to n , $1 \leq s \leq n - 1$, computes $v = s^2 \bmod n$, and registers v with T as her public key.

Fiat-Shamir Protocol

Identification Protocol

The following steps are executed t times, each time using independent random coin tosses.

- (P1) The prover chooses a random r , $1 \leq r \leq n - 1$ and sends $x = r^2 \bmod n$ to the verifier.
- (V1) The verifier randomly selects a bit $e \in \{0, 1\}$ and sends e to the prover.
- (P2) The prover computes and sends to the verifier y , where $y = r$ (if $e = 0$) or $y = rs \bmod n$ (if $e = 1$).
- (V2) The verifier *rejects* if $y = 0$ or if $y^2 \not\equiv x \cdot v^e \pmod{n}$. (Depending on e , $y^2 = x$ or $y^2 = xv \bmod n$, since $v = s^2 \bmod n$. Note that checking for $y = 0$ precludes the case $r = 0$.)

Fiat-Shamir Protocol

Remark 5.3. As in the cave story, one might wonder why Victor doesn't always request $y = rs$, thus forcing Peggy to make use of s . If this is done, Peggy has a cheating strategy. She can select r randomly and then set $x = \frac{r^2}{v}$. When Victor requests $y = rs$, Peggy will send $y = r$. Now, when Victor attempts to verify Peggys response, he will compute:

$$x \cdot v^e \equiv \frac{r^2}{v} \cdot v \equiv r^2 \equiv y^2 \pmod{n}$$

thus fooling Victor into believing she has possession of s . This strategy becomes ineffective if Victor asks either of the two questions at random.

Fiat-Shamir Protocol

Example 5.4 (An example run of the Fiat-Shamir Identification Protocol).

Initialization

1. Let $p = 5$ and $q = 7$. Then, $n = pq = 35$. n is published to a trusted center.
2. Peggy secretly chooses $s = 16$, which is coprime to n . She publishes $v = s^2 \bmod n = 11$ to the trusted center.

Fiat-Shamir Protocol

Protocol

Suppose Victor is very easy to convince, and hence requires only 2 successful iterations of the protocol in order to *accept*.

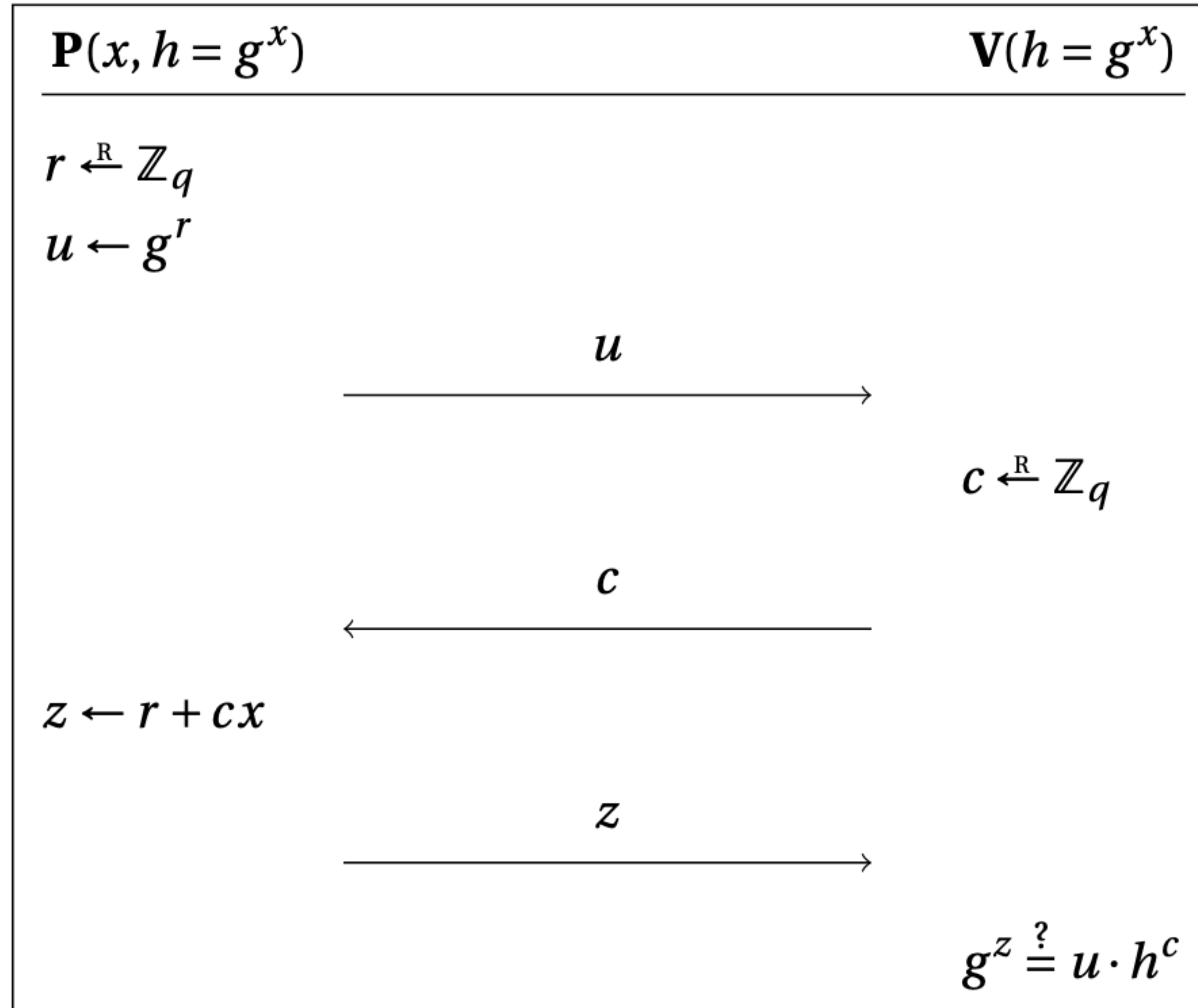
- (P1₁) Peggy randomly selects $r = 10$. She sends $x = r^2 \bmod n = 10^2 \bmod 35 = 30$ to Victor.
- (V1₁) Victor randomly selects $e = 0$ and sends it to Peggy.
- (P2₁) Since $e = 0$, Peggy computes $y = r = 10$ and sends it to Victor.
- (V2₁) Victor verifies that $y^2 = 10^2 \equiv 30 \pmod{35}$.
- (P1₂) Peggy randomly selects $r = 20$. She sends $x = r^2 \bmod n = 20^2 \bmod 35 = 15$ to Victor.

Fiat-Shamir Protocol

- (V1₂) Victor randomly selects $e = 1$ and sends it to Peggy.
- (P2₂) Since $e = 1$, Peggy computes $y = sr \bmod n = 16 \cdot 20 \bmod 35 = 5$ and sends it to Victor.
- (V2₂) Victor verifies that $y^2 = 25 \equiv 15 \cdot 11 \pmod{35}$.

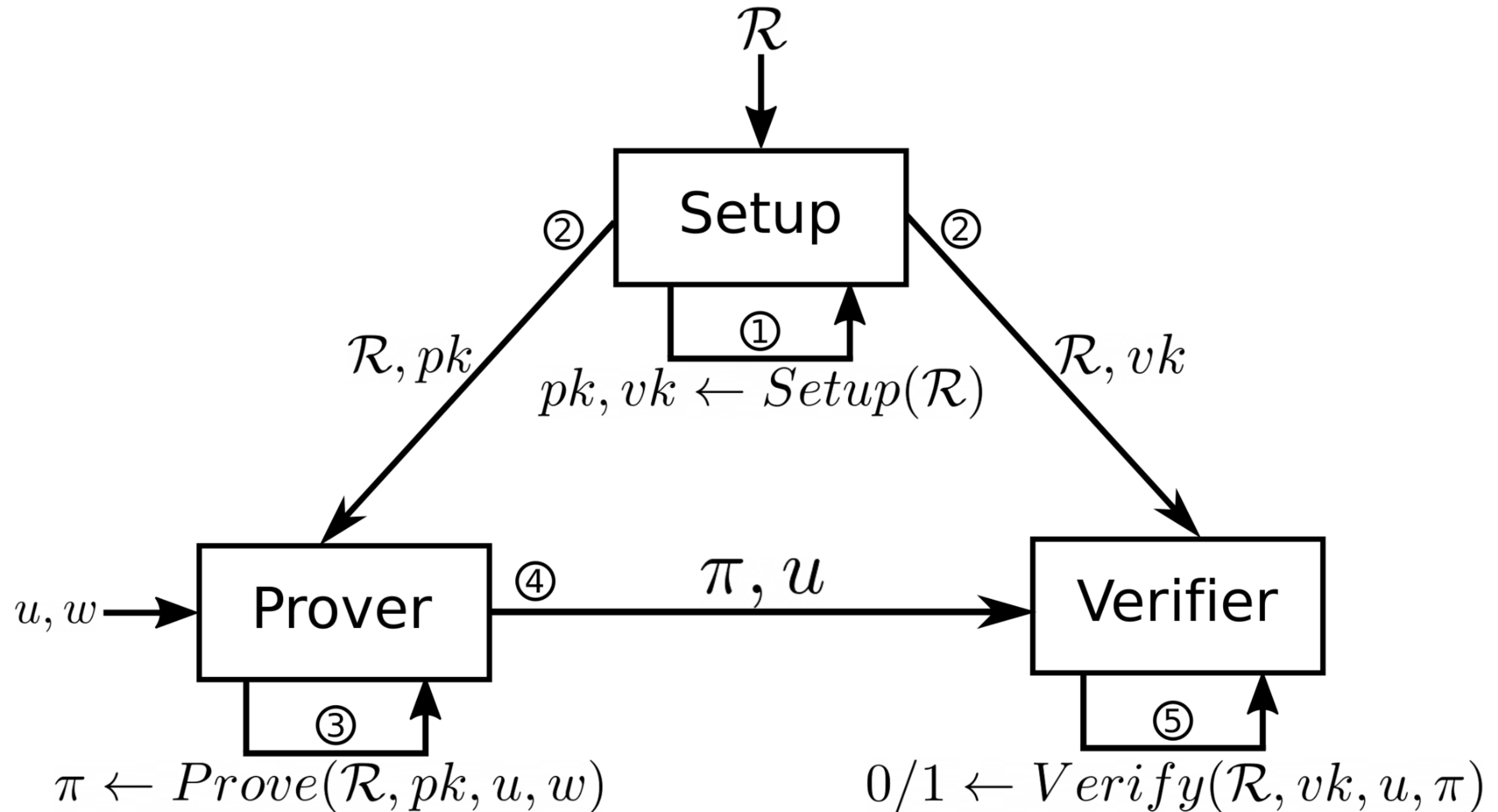
Schnorr's Protocol

Discrete Log



$$g^z \stackrel{?}{=} u \cdot h^c$$

Zero-Knowledge Proof



ZoKrates - Scalable Privacy-Preserving Off-Chain Computations

Jacob Eberhardt, Stefan Tai
Information Systems Engineering (ISE)
TU Berlin
Berlin, Germany
{je, st}@ise.tu-berlin.de

Abstract—Scalability and privacy are two major challenges for today’s blockchain systems. Processing transactions at every node in the system limits the system’s ability to scale. Further, the requirement to publish all corporate or individual information for processing at every node, essentially making the data public, is – despite of all other advantages – often considered a major obstacle to blockchain adoption. In this paper, we make two main contributions to address these two problems: (i) To increase efficiency, we propose a processing model which employs non-interactive proofs to off-chain computations, thereby reducing on-chain computational efforts to the verification of correctness of execution rather than the execution itself. Due to the verifiable computation scheme’s zero-knowledge property, private information used in the off-chain computation does not have to become public to verify correctness. (ii) We introduce ZoKrates, a toolbox to specify, integrate and deploy such off-chain computations. It consists of a domain-specific language, a compiler, and generators for proofs and verification Smart Contracts. ZoKrates hides significant complexity inherent to zero-knowledge proofs, provides a more familiar and higher level of programming abstractions to developers and enables circuit integration, hence fostering adoption.

Index Terms—ZoKrates, zkSNARKs, off-chain, scalability, privacy

increase transaction throughput by reducing the cost associated with transaction validation; (b) through off-chaining, remove the requirement to reveal data to all and to become public but allow data to remain private, yet, private data must still have an influence on the transactions processed on-chain.

Transaction throughput is directly linked to the cost implied for clients for block validation. While only mining nodes spend resources on proof of work and are economically incentivized to do so, every full node in the network needs to process all occurring transactions to ensure validity without receiving compensation. To ensure an upper bound for the cost of this validation process, termination is guaranteed. Bitcoin limits transaction complexity by employing a non-turing-complete scripting language and a block size limit. Ethereum, in contrast, uses the concept of gas as a measure for how much burden is put on clients by a transaction (in terms of computation and storage) and defines an upper block gas limit.

In this paper, we make two main contributions:

- 1) First, we introduce an off-chain processing model based on non-interactive zero-knowledge proofs to improve

ZoKrates: Contributions

- 1) First, we introduce an off-chain processing model based on non-interactive zero-knowledge proofs to improve on transaction throughput and privacy for blockchain systems. In this model, computations are performed off-chain and afterwards the result is written to the blockchain. With the result, a proof attesting correctness of the off-chain execution is supplied. This is then validated on-the blockchain. To realize the benefits mentioned above, the proof system needs to fulfill two important criteria: a) Proof verification needs to be faster than (re-)execution of the original computation. b) The proof needs to be zero-knowledge, that means, it should be possible to proof statements on private data without revealing it. Combining a suitable proof system with the off-chain processing model, we provide means to process blockchain transactions in a more scalable and privacy-preserving way.

ZoKrates: Contributions

- 2) There are suitable cryptographic verifiable computation schemes, but applying them to blockchains is hard. Computations need to be specified in difficult-to-use low level abstractions. On-chain verification is very complex and requires deep knowledge of the employed schemes. As a solution, we introduce ZoKrates, a set of software tools to support the off-chain processing model above by employing zkSNARKs as a proof system. ZoKrates defines a domain-specific language which allows developers to conveniently specify off-chain computations at a high level of abstraction. This enables them to specify provable computations without having to understand the proof system's low-level programming abstractions. For that purpose, ZoKrates includes a compiler which translates domain-specific code into provable constraint systems. It furthermore assists in executing off-chain programs and generating proofs attesting their correctness. To conveniently enable on-chain verification, ZoKrates supports the export of verification Smart Contracts, which verify the proofs generated off-chain and therefore confirm the off-chain computation's correctness. Our proof-of-concept implementation targets the Ethereum blockchain, but is by design compatible with other blockchain systems.

ZoKrates: Off-Chaining

Taking computation and data off the blockchain can have significant benefits for performance, cost and privacy [3]. While anchoring cryptographic hashes to the blockchain showed to be a suitable way of referring to off-chain data [10], [11], moving computations from the blockchain to external nodes without compromising its properties is a more challenging problem.

Delegating computation to arbitrary nodes which publish results to the blockchain can have two main benefits when compared to on-chain execution:

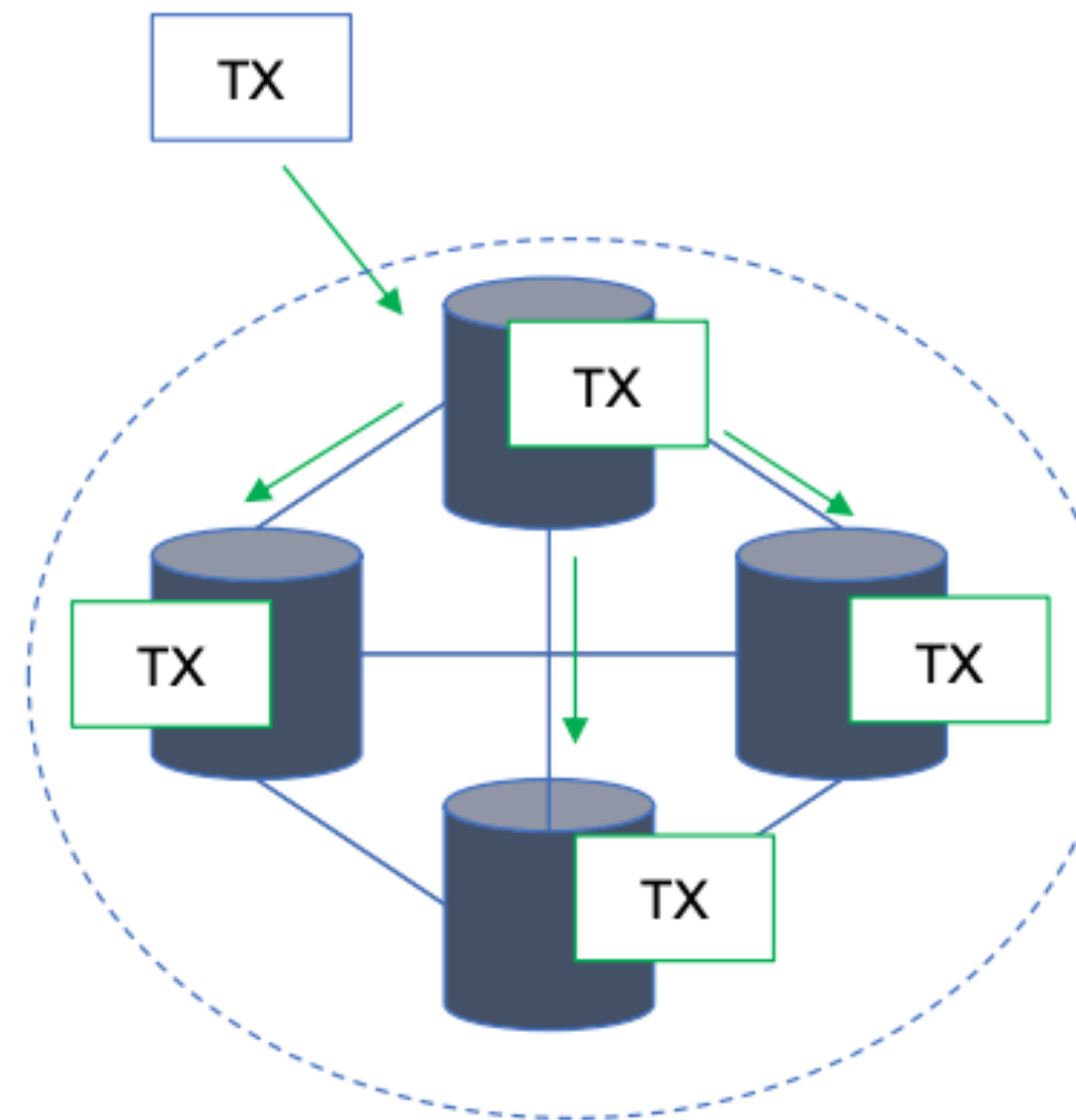
- 1) A delegate node can use private information during execution of a computation without publicly revealing it. This is not possible for on-chain computations, since they are performed redundantly on all nodes in the network which therefore need to be in possession of all data involved.
- 2) Ideally, a delegate would only write the result of a computation to the blockchain, which would highly increase efficiency as the only operation performed redundantly by each node would be storing the result. This would largely increase throughput, as many more state transitions could happen in one block.

ZoKrates: Off-Chaining

For this to be viable in practice, the verifiable computation scheme needs to have the following key properties:

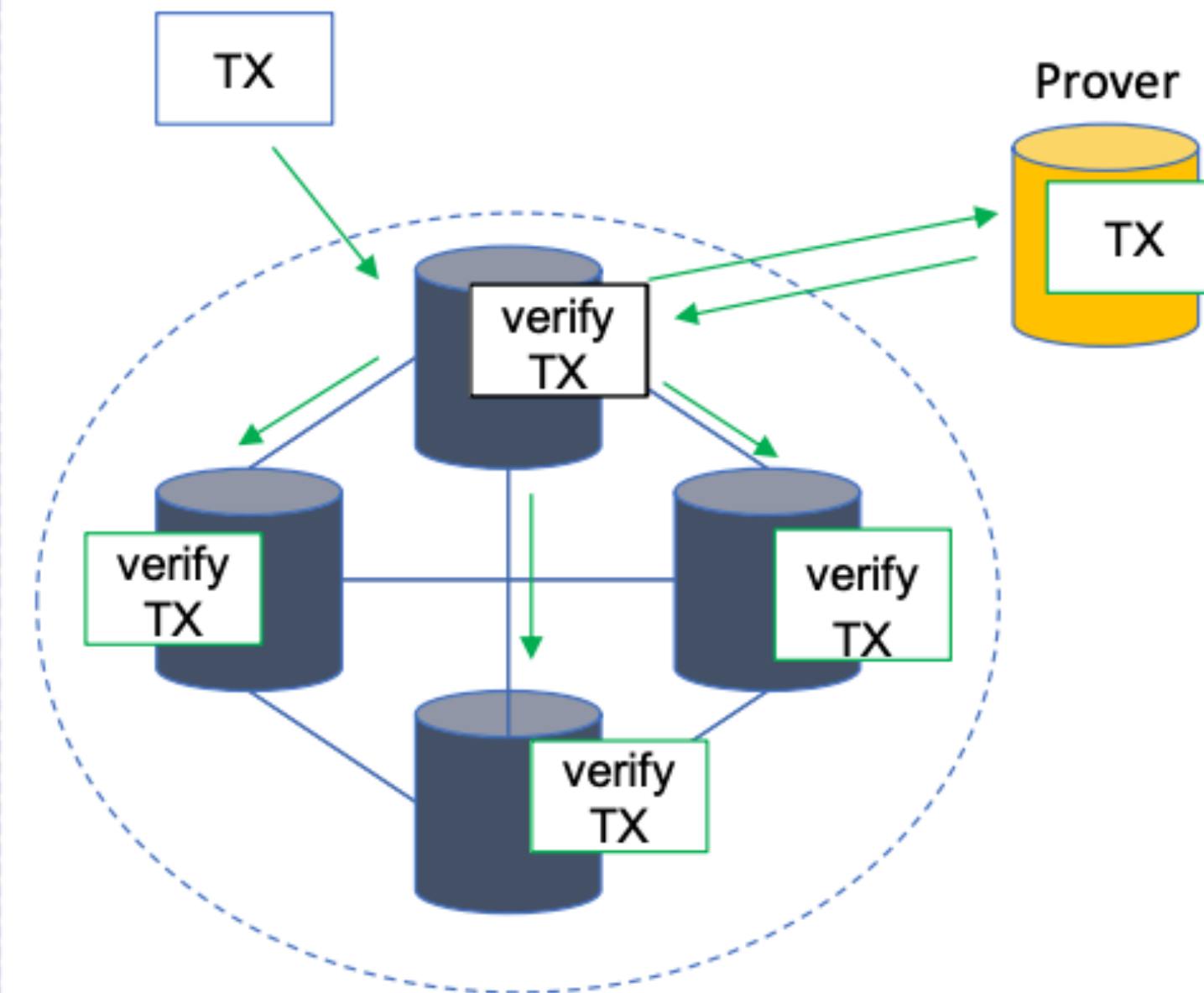
- Short proofs and cheap verification.
- Zero-knowledge, i.e., the verifier learns nothing but the fact that the computation was executed correctly.

On-chain processing



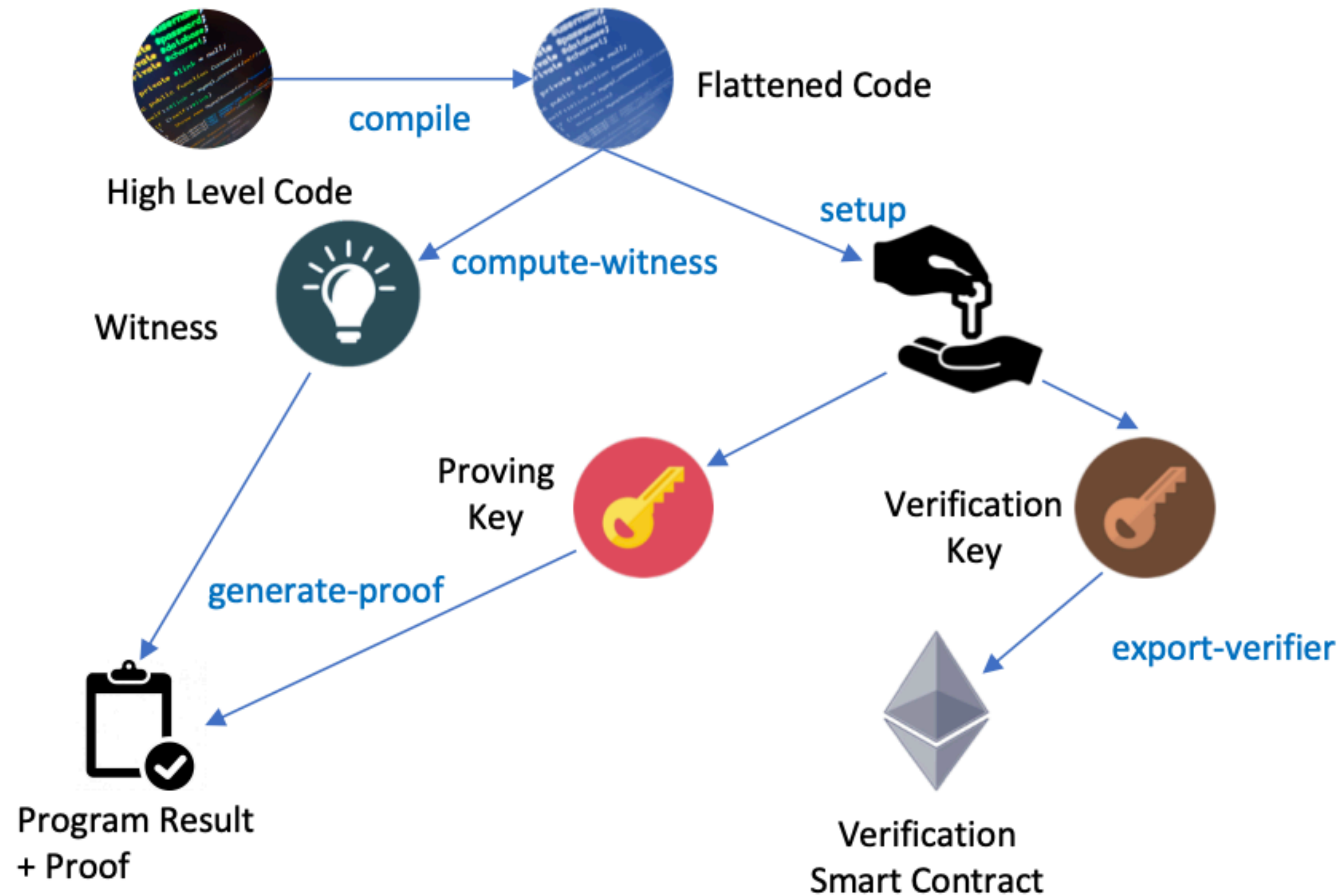
Blockchain Network

Delegated computation

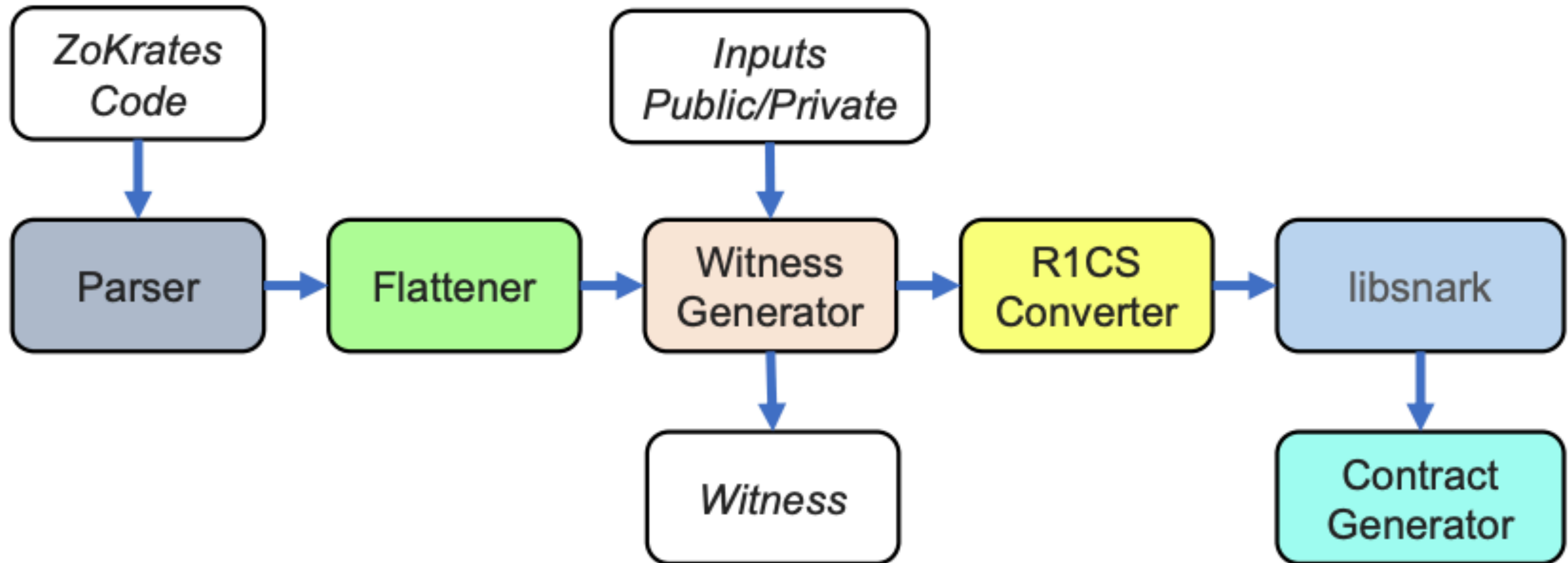


Blockchain Network

ZoKrates: Architecture

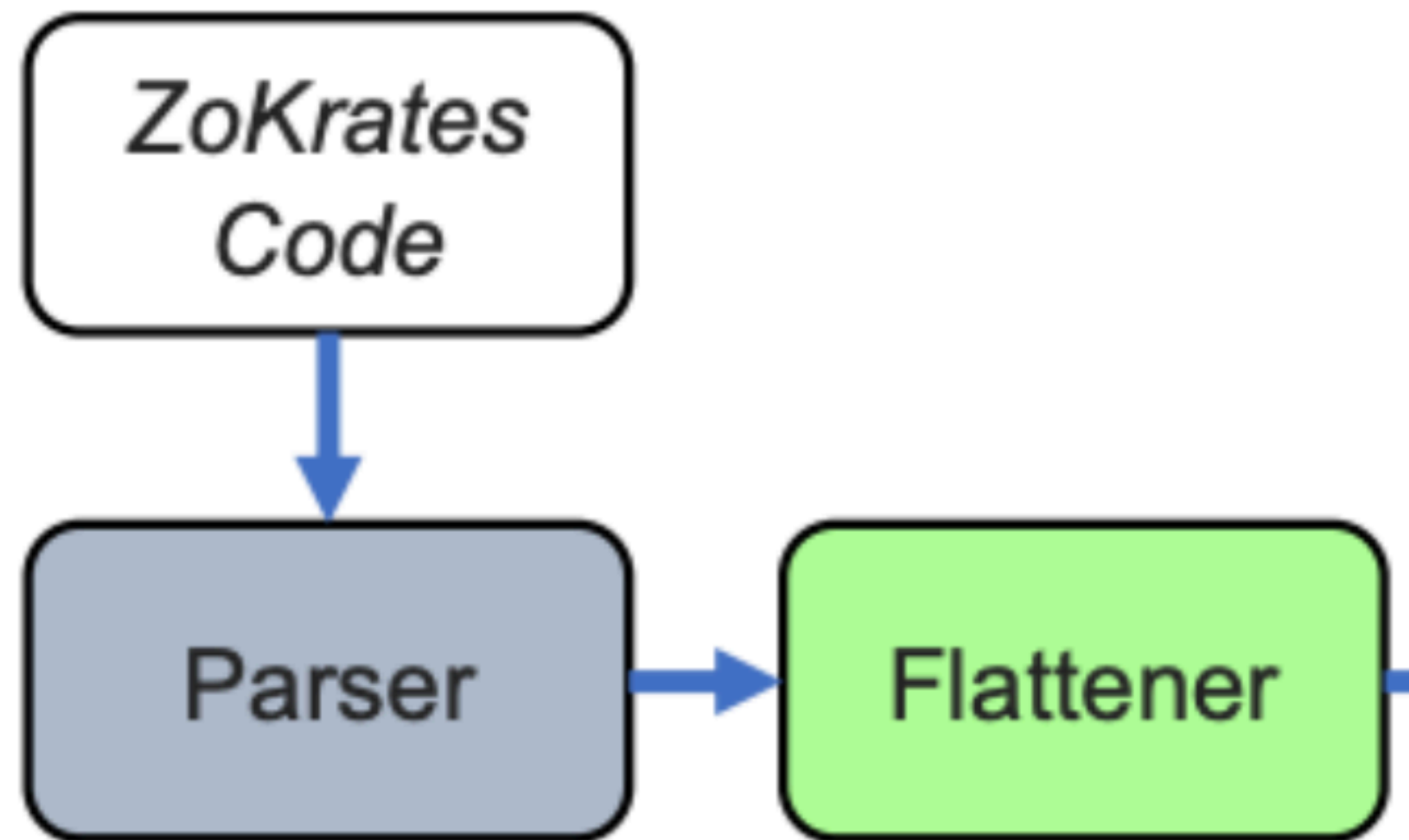


ZoKrates: Architecture



ZoKrates: Architecture

1) *Compiler*: The compiler, consisting of the parser and the flattener in Fig. 3, translates DSL code into so-called flattened code. Flattened code consists of a list of variable definitions and assertions of the following form:

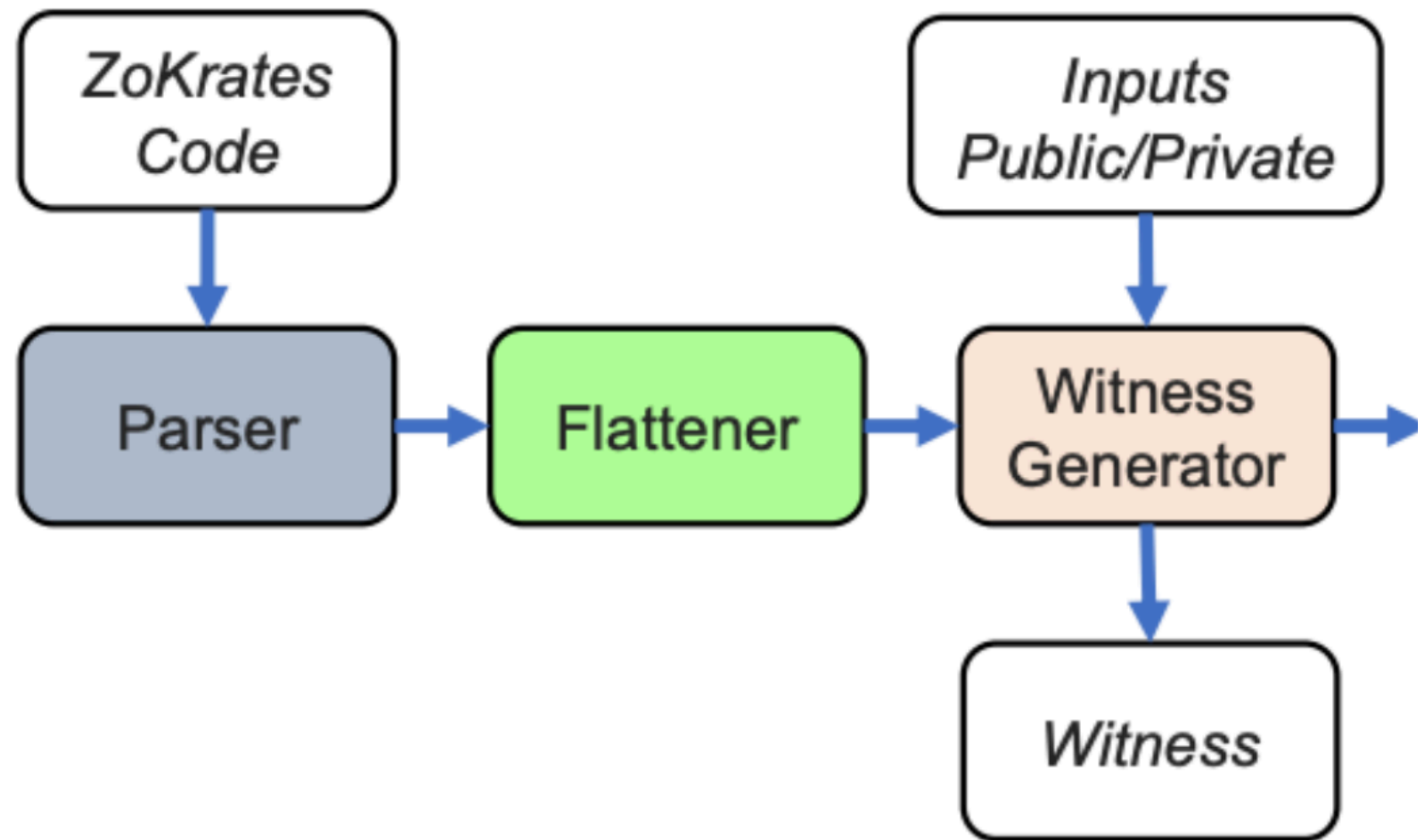


$$x^3 + x + 5 = 35$$

```
def qeval(x):  
    y = x**3  
    return x + y + 5
```

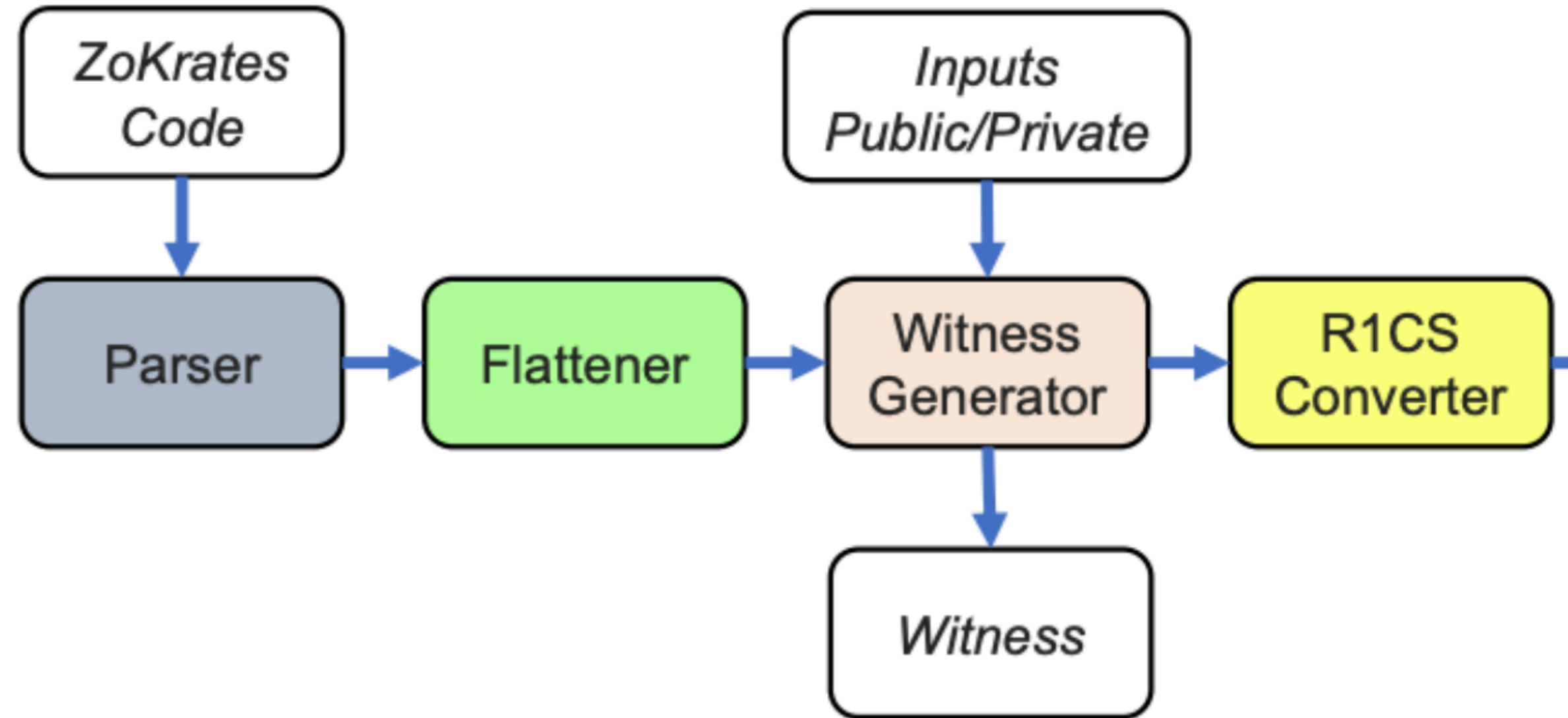
```
sym_1 = x * x  
y = sym_1 * x  
sym_2 = y + x  
~out = sym_2 + 5
```

ZoKrates: Architecture



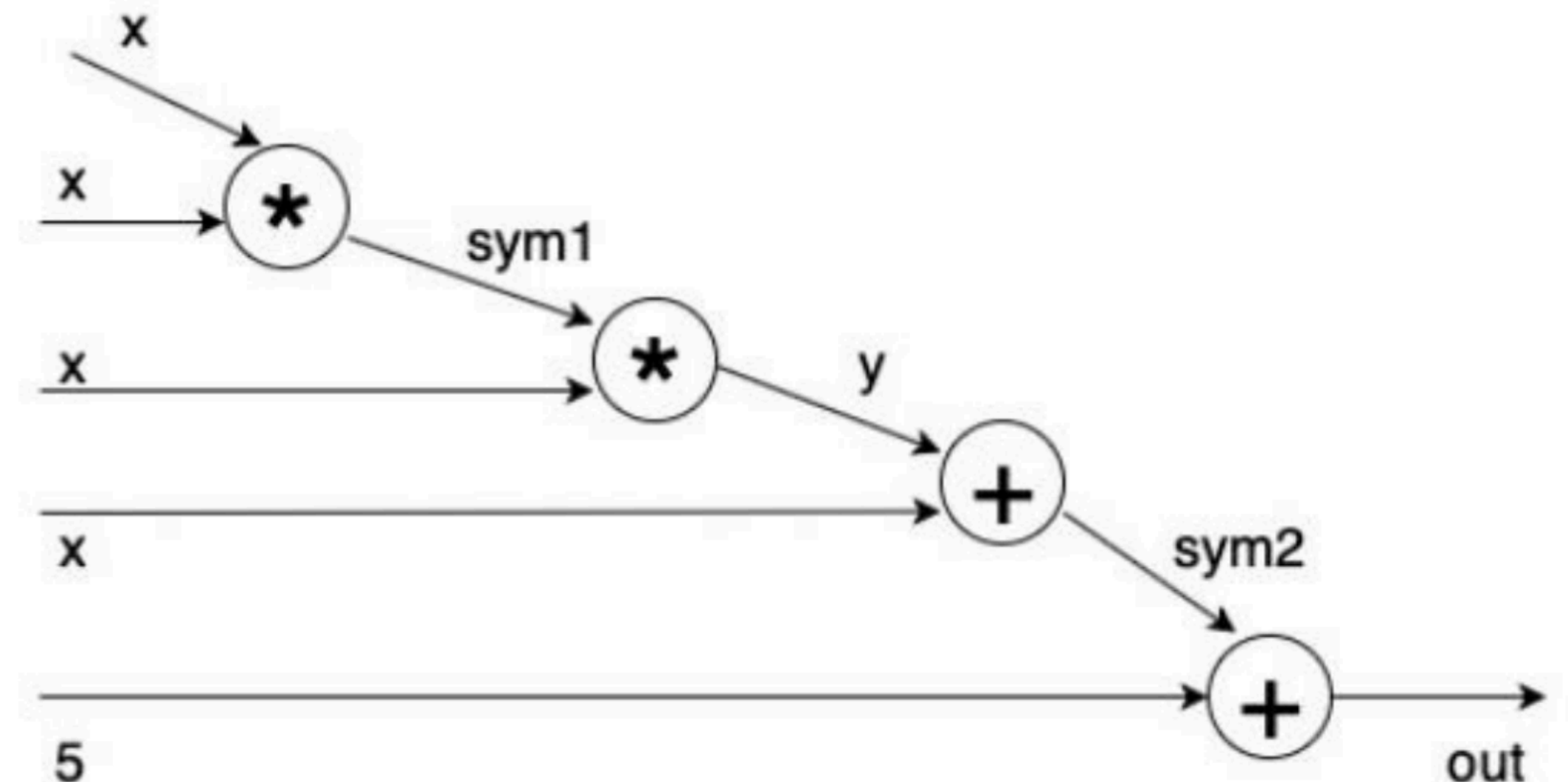
2) *Witness Generator*: Before a proof attesting correct execution can be generated, the program first has to be executed. In this context, that means finding a satisfying variable assignment for a flattened program. Such a variable assignment is referred to as a witness and can then be used during proof generation. A witness for a given program can be found using the witness generator which acts as an interpreter for flattened code. It takes a program's private inputs as parameters, executes the flattened code and stores all variable values, thus finding a witness in the process.

ZoKrates: Architecture

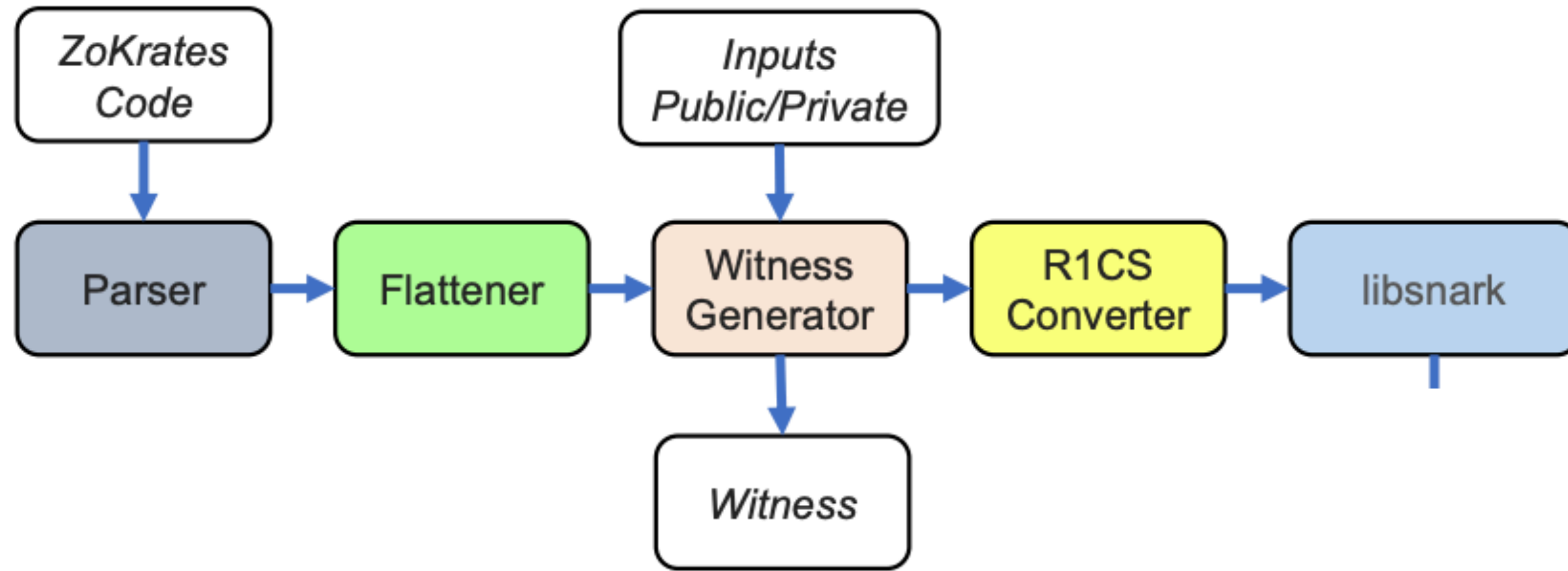


```
sym_1 = x * x
y = sym_1 * x
sym_2 = y + x
~out = sym_2 + 5
```

3) *Circuit Import*: In order to be able to leverage hand-optimized circuits for certain computations or integrate with other circuit generation tools (see section VII), R1CS can be imported into semantically equivalent flattened code. However, while fully supported for ZoKrates DSL code, witness generation for imported programs cannot generally be supported and requires users to provide that information.



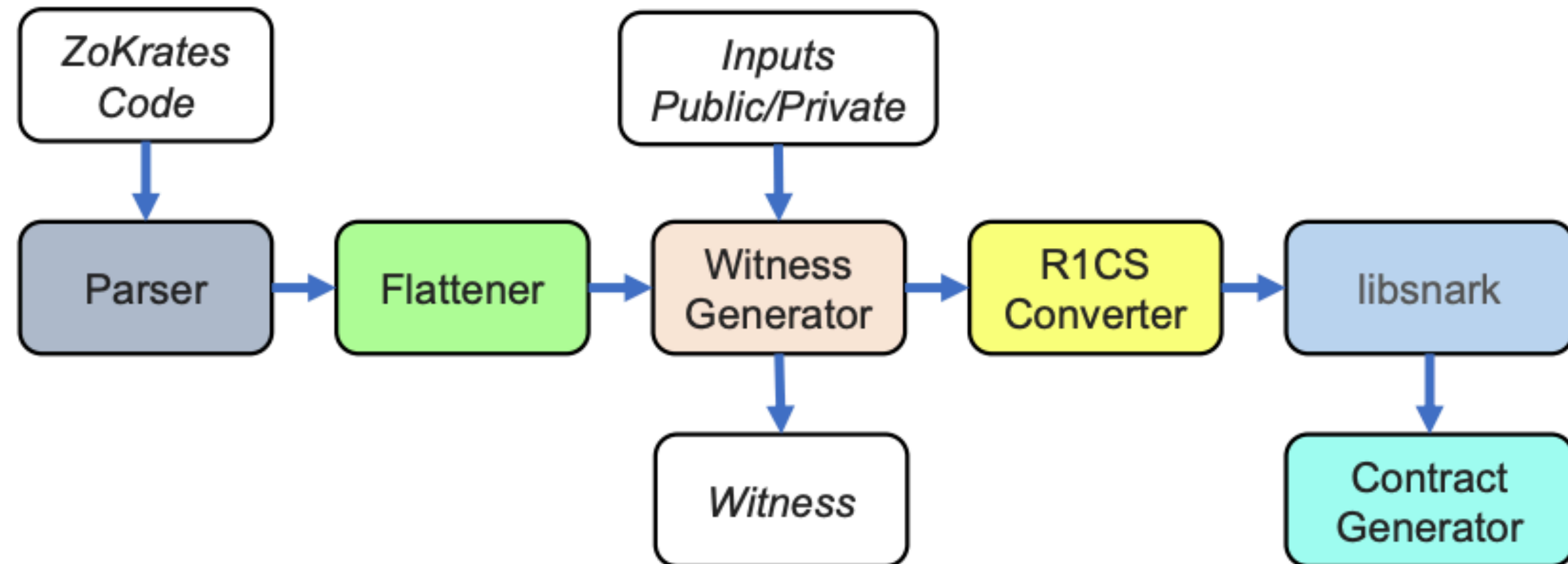
ZoKrates: Architecture



4) *Setup and Proof Generation:* As explained in section VII, the zkSNARK verifiable computation scheme requires a one-time trusted setup to be performed. It is trusted in the sense that who executes it needs to forget private information used in the process. If that information is not forgotten, the owner could potentially create fake proofs. However, the zero-knowledge property of other proofs would not be impaired. To address this issue, we discuss mechanisms to remove this trust in section VI. The setup phase takes a program and its public arguments as an input and generates a proving and a verification key which can then be used to create and verify proofs [13]. These keys can then be reused an arbitrary number of times. For setup and proof generation, the ZoKrates implementation uses `libsark`⁴, a cryptographic library which implements zkSNARK schemes [14].

5) *Proof Generation:* After a witness has been found and the proving key generated, a proof attesting the correctness of the witness and with that program execution can be created. The resulting proof is very short (see V for details) and can thus be efficiently sent to the blockchain via network.

ZoKrates: Architecture



6) *Contract Generator*: Based on a verification key, we generate a Solidity Smart Contract which can verify proofs on the blockchain. The generated contract has a very simple base structure as shown in Listing 4 and can be customized depending on the concrete use case the on-chain verification is leveraged in. It can be compiled with `solc`⁵, the official Solidity compiler, and then be deployed to Ethereum. The `verifyTx` function takes a program's public inputs and results as parameters as well as the previously generated proof attesting correctness of its execution. It returns a boolean value indicating verification success and hence the correctness of the off-chain execution. If successful, the program's result is confirmed and can be used in further processing on or off the blockchain.

ZoKrates: Architecture

```
contract Verifier {
    // library wrapping precompiles
    using Pairing for *;

    // verification key data structure
    struct VerifyingKey {
        ...
    }

    // proof data structure
    struct Proof {
        ...
    }

    event Verified(string);

    // Accepts proof and public inputs and checks
    // proof validity.
    // Proof: a, a_p, b, b_p, c, c_p, h, k.
    function verifyTx(
        uint[2] a,
        uint[2] a_p,
        uint[2][2] b,
        uint[2] b_p,
        uint[2] c,
        uint[2] c_p,
        uint[2] h,
        uint[2] k,
        uint[] inputs_and_res
    ) returns (bool r) {
        ...
    }
}
```

ZoKrates DSL: Variables

Declaration

Variables need to be declared to be used. Declaration and definition are always combined, so that undefined variables do not exist.

```
def main() {  
    // declare and define `my_variable`  
    field mut my_variable = 2;  
    // redefine `my_variable`  
    my_variable = 3;  
    return;  
}
```



ZoKrates DSL: Variables

Mutability

Variables are immutable by default. In order to declare a mutable variable, the `mut` keyword is used.

```
def main() {  
  field a = 42;  
  // a = 43; <- not allowed, as `a` is immutable  
  field mut b = 42;  
  b = 43; // ok  
  return;  
}
```



ZoKrates DSL: Variables

Shadowing

Shadowing is allowed.

```
def main() -> field {  
    field a = 2;  
    field a = 3; // shadowing  
    for u32 i in 0..5 {  
        bool a = true; // shadowing  
    }  
    // `a` is the variable declared before the loop  
    return a;  
}
```



ZoKrates DSL: Variables

Function

Functions have their own scope

```
def foo() -> field {  
    // return myGlobal; <- not allowed  
    return 42;  
}  
  
def main() -> field {  
    field myGlobal = 42;  
    return foo();  
}
```



ZoKrates DSL: Variables

For-loop

For-loops have their own scope

```
def main() -> u32 {  
    u32 mut a = 0;  
    for u32 i in 0..5 {  
        a = a + i;  
    }  
    // return i; <- not allowed  
    return a;  
}
```



ZoKrates DSL: Types

`field`

This is the most basic type in ZoKrates, and it represents a field element with positive integer values in `[0, p - 1]` where `p` is a (large) prime number.

As an example, `p` is set to

`21888242871839275222246405745257275088548364400416034343698204186575808495617` when working with the [ALT_BN128](#) curve supported by Ethereum.

While `field` values mostly behave like unsigned integers, one should keep in mind that they overflow at `p` and not some power of 2, so that we have:

```
def main() {  
    field pMinusOne = 2188824287183927522224640574525727508854836440041603434369820418657  
    assert(0 - 1 == pMinusOne);  
    return;  
}
```



ZoKrates DSL: Types

`bool`

Booleans are available in ZoKrates. When a boolean is used as a parameter of the main function, the program is constrained to only accept `0` or `1` for that parameter. A boolean can be asserted to be true using an `assert(bool)` statement.

ZoKrates DSL: Types

`u8/u16/u32/u64`

Unsigned integers represent positive numbers of the interval `[0, 2 ** bitwidth)`, where `bitwidth` is specified in the type's name, e.g., 32 bits in the case of `u32`. Their arithmetics are defined modulo `2 ** bitwidth`.

Internally, they use a binary encoding, which makes them particularly efficient for implementing programs that operate on that binary representation, e.g., the SHA256 hash function.

Similarly to booleans, unsigned integer inputs of the main function only accept values of the appropriate range.

The division operation calculates the standard floor division for integers. The `%` operand can be used to obtain the remainder.

ZoKrates DSL: Types

Arrays

ZoKrates supports static arrays, i.e., whose length needs to be known at compile time. For more details on generic array sizes, see [constant generics](#). Arrays can contain elements of any type and have arbitrary dimensions.

The following example code shows examples of how to use arrays:

```
def main() -> field {  
    field[3] mut a = [1, 2, 3]; // initialize a field array with field values  
    a[2] = 4;                  // set a member to a value  
    field[4] b = [42; 4];      // initialize an array of 4 values all equal to 42  
    field[4] c = [...a, 4];    // initialize an array copying values from `a`, followed by 4  
    field[2] d = a[1..3];      // initialize an array copying a slice from `a`  
    bool[3] e = [true, true || false, true]; // initialize a boolean array  
    u32 SIZE = 3;  
    field[SIZE] f = [1, 2, 3]; // initialize a field array with a size that's a compile-t  
    return a[0] + b[1] + c[2];  
}
```

ZoKrates DSL: Types

Spreads

The spread operator `...` applied to an array copies the elements of the existing array. This can be used to conveniently compose new arrays, as shown in the following example:

```
field[3] a = [1, 2, 3];  
field[4] c = [...a, 4]; // initialize an array copying values from `a`, followed by 4
```



Slices

An array can also be assigned to by creating a copy of a subset of an existing array. This operation is called slicing, and the following example shows how to slice in ZoKrates:

```
field[3] a = [1, 2, 3];  
field[2] b = a[1..3]; // initialize an array copying a slice from `a`
```



ZoKrates DSL: Types

Multidimensional Arrays

As an array can contain any type of elements, it can contain arrays again. There is a special syntax to declare such multi-dimensional arrays, i.e., arrays of arrays. To declare an array of an inner array, i.e., and an array of elements of a type, prepend brackets `[size]` to the declaration of the inner array. In summary, this leads to the following scheme for array declarations:

`data_type[size of 1st dimension][size of 2nd dimension]`. Consider the following example:

```
def main() -> field {  
    // Array of two elements of array of 3 elements  
    field[2][3] a = [[1, 2, 3],[4, 5, 6]];  
  
    field[3] b = a[0]; // should be [1, 2, 3]  
  
    // allowed access [0..2][0..3]  
    return a[1][2];  
}
```



ZoKrates DSL: Types

Tuples

A tuple is a composite datatype representing a numbered collection of values. The following code shows an example of how to use tuples.

```
def main() -> bool {  
    (field[2], bool) mut v = ([1, 2], true);  
    v.0 = [42, 43];  
    return v.1;  
}
```



ZoKrates DSL: Types

Structs

Before a struct data type can be used, it needs to be defined. A struct definition starts with the `struct` keyword followed by a name. Afterwards, a new-line separated list of variables is declared in curly braces `{}`. For example:

```
struct Point {  
    field x;  
    field y;  
}
```



Note that two struct definitions with the same members still introduce two entirely different types. For example, they cannot be compared with each other.

ZoKrates DSL: Types

Structs

Before a struct data type can be used, it needs to be defined. A struct definition starts with the `struct` keyword followed by a name. Afterwards, a new-line separated list of variables is declared in curly braces `{}`. For example:

```
struct Point {  
    field x;  
    field y;  
}
```



ZoKrates DSL: Control Flow

Function calls

Function calls help make programs clear and modular.

Arguments are passed by value.

Function parameters can be declared as mutable to allow for mutation within the function's body. However, mutable function arguments are still passed by value, so the original value can never be mutated.

```
def incr(field mut a) -> field {  
    a = a + 1;  
    return a;  
}  
  
def main() {  
    field x = 1;  
    field res = incr(x);  
    assert(x == 1); // x has not changed  
    return;  
}
```



ZoKrates DSL: Control Flow

Conditional expressions

A conditional expression allows you to branch your code depending on a boolean condition.

```
def main(field x) -> field {  
    field y = if x + 2 == 3 { 1 } else { 5 };  
    return y;  
}
```



The conditional expression can also be written using a ternary operator:

```
def main(field x) -> field {  
    field y = x + 2 == 3 ? 1 : 5;  
    return y;  
}
```



ZoKrates DSL: Control Flow

Now the two caveats:

- **Both branches are always executed.** No short-circuiting happens based on the value of the condition. Therefore, the complexity of a program in terms of the number of constraints it compiles down to is the *sum* of the cost of all branches.

```
def cheap(field x) -> field {  
    return x + 1;  
}  
  
def expensive(field x) -> field {  
    return x**1000;  
}  
  
def main(field x) -> field {  
    return if x == 1 {  
        cheap(x)  
    } else {  
        expensive(x) // both branches executed  
    };  
}
```

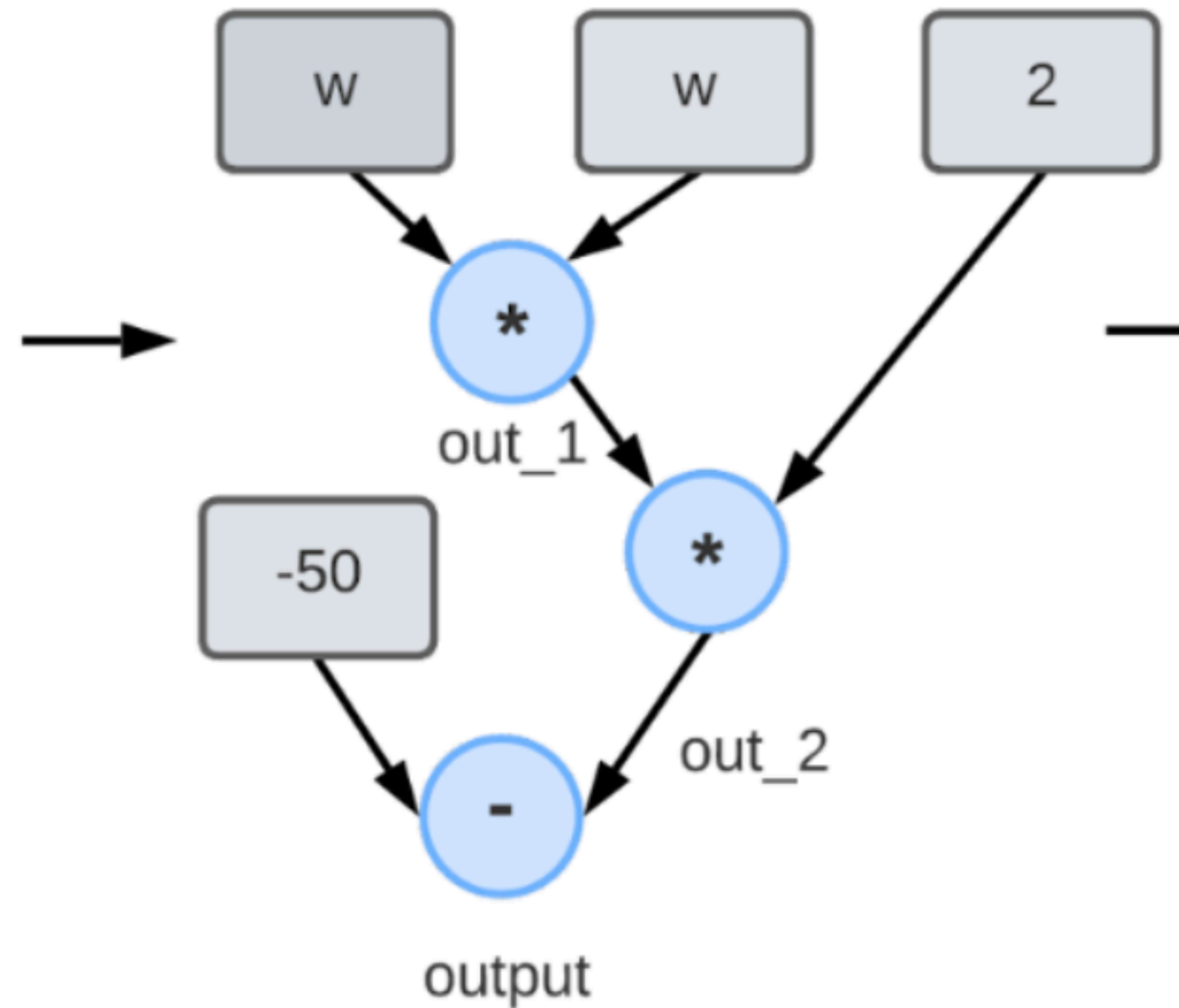


ZoKrates DSL: Control Flow

Program

```
function {  
  return 2*(w^2) == 50;  
}
```

Arithmetization



ZoKrates DSL: Control Flow

For loops

For loops are available with the following syntax:

```
def main() -> u32 {  
    u32 mut res = 0;  
    for u32 i in 0..4 {  
        for u32 j in i..5 {  
            res = res + i;  
        }  
    }  
    return res;  
}
```



ZoKrates DSL: Control Flow

Assertions

Any boolean can be asserted to be true using the `assert` function.

```
def main() {  
    assert(1f + 1f == 2f);  
    return;  
}
```



If any assertion fails, execution stops as no valid proof could be generated from it.

ZoKrates DSL: Constants

Constants

Constants must be globally defined outside all other scopes by using a `const` keyword. Constants can be set only to a constant expression.

```
const field PRIME = 31;

def main() -> field {
    return PRIME;
}
```



The value of a constant can't be changed through reassignment, and it can't be redeclared.

Constants must be explicitly typed. One can reference other constants inside the expression, as long as the referenced constant is already defined.

```
const field ONE = 1;
const field TWO = ONE + ONE;

def main() -> field {
    return TWO;
}
```



The naming convention for constants are similar to that of variables. All characters in a constant name are usually in uppercase.

ZoKrates DSL: Imports

Import syntax

Symbol selection

The preferred way to import a symbol is by module and name:

```
from "../path/to/my/module" import MySymbol;  
  
// `MySymbol` is now in scope.
```



ZoKrates DSL: Comments

```
def main() -> field {  
    field a = 42; // this is an end of line comment  
    // this is a full line comment  
    return a;  
}
```



```
/*  
    This is a multi-line comment  
    written in more than just one line.  
*/  
def main() -> field {  
    return 42;  
}
```



ZoKrates DSL: Hashes

SHA256

We provide an implementation of the SHA256 function from the SHA-2 family of secure hash functions ¹. The hash functions of the SHA-2 family are considered to be pseudorandom.

SHA256 is available in Ethereum as a pre-compiled contract and thus a hash function that is cheap to evaluate in the EVM. However, the implementation inside a circuit is comparatively expensive, as it is defined for binary in- and outputs and heavily relies on bit manipulation.

```
def main(u32[8] a, u32[8] b) -> u32[8] {  
    return sha256([[...a, ...b]]);  
}
```

ZoKrates DSL: CLI

We provide one-line installation for Linux, MacOS and FreeBSD:

```
curl -LSfs get.zokrat.es | sh
```

```
# compile
zokrates compile -i root.zok
# perform the setup phase
zokrates setup
# execute the program
zokrates compute-witness -a 337 113569
# generate a proof of computation
zokrates generate-proof
# export a solidity verifier
zokrates export-verifier
# or verify natively
zokrates verify
```

ZoKrates DSL: JavaScript

JavaScript bindings for [ZoKrates](#).

```
npm install zokrates-js
```

```
initialize().then((zokratesProvider) => {  
  const source = "def main(private field a) -> field { return a * a; }";  
  
  // compilation  
  const artifacts = zokratesProvider.compile(source);  
  
  // computation  
  const { witness, output } = zokratesProvider.computeWitness(artifacts, ["2"]);  
  
  // run setup  
  const keypair = zokratesProvider.setup(artifacts.program);  
  
  // generate proof  
  const proof = zokratesProvider.generateProof(  
    artifacts.program,  
    witness,  
    keypair.pk  
  );  
  
  // export solidity verifier  
  const verifier = zokratesProvider.exportSolidityVerifier(keypair.vk);  
  
  // or verify off-chain  
  const isVerified = zokratesProvider.verify(keypair.vk, proof);  
});
```

ZoKrates DSL: Random Number

Description of the problem

Alice and Bob want to bet on the result of a series of coin tosses. To do so, they need to generate a series of random bits. They proceed as follows:

1. Each of them commits to a 512 bit value. Let's call this value the ***preimage***. They publish the hash of the preimage.
2. Each time they need a new random value, they reveal one bit from their preimage, and agree that the new random value is the result of XORing these two bits, so that neither of them can control the output.

ZoKrates DSL: Random Number

Commit to a preimage

The first step is for Alice and Bob to each come up with a preimage value and calculate the hash to commit to it. There are many ways to calculate a hash, but here we use ZoKrates.

Create this file under the name `get_hash.zok`:

```
import "hashes/sha256/512bit" as sha256;

def main(u32[16] hashMe) -> u32[8] {
    u32[8] h = sha256(hashMe[0..8], hashMe[8..16]);
    return h;
}
```



ZoKrates DSL: Pre-image Knowledge

Prove knowledge of pre-image

For now, we have seen that we can compute a hash using ZoKrates.

Let's recall our goal: Peggy wants to prove that she knows a preimage for a digest chosen by Victor, without revealing what the preimage is. Without loss of generality, let's now assume that Victor chooses the digest to be the one we found in our example above.

```
import "hashes/sha256/512bitPacked" as sha256packed;

def main(private field a, private field b, private field c, private field d) {
    field[2] h = sha256packed([a, b, c, d]);
    assert(h[0] == 263561599766550617289250058199814760685);
    assert(h[1] == 65303172752238645975888084098459749904);
    return;
}
```